LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

10

Linux Shell Scripting and Automation

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 What Is a Shell?
02 Your First Bash Script
03 Variables and Quoting
04 Taking User Input
05 Conditions and case
06 Loops and Arrays
07 Functions
08 Exit Codes
09 Error Handling and Debugging
10 Command-Line Arguments
11 Redirection and Logging
12 DevOps Automation Scripts
13 Scheduling with Cron
14 Best Practices
15 Troubleshooting Scenarios
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar
Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux
github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Learn how to automate repetitive Linux administration tasks using Bash scripting. Understand variables, user input, conditions, loops, functions, exit codes, error handling and scheduling. Build practical scripts for backups, disk monitoring, service checks and DevOps automation.

1 What Is a Shell?

A shell is a command-line interface that allows users to interact with the Linux operating system. When you execute a command in a terminal, the shell interprets it and communicates with the operating system to perform the requested operation.

Shell	Description
Bash	Bourne Again Shell; widely used in Linux
sh	Traditional Unix shell interface
Zsh	Interactive shell with advanced features
Fish	User-friendly interactive shell

```
# Check your current shell
echo $SHELL

# Check available shells
cat /etc/shells

# Check the installed Bash version
bash --version
```

What Is Bash Scripting?

Bash scripting means writing multiple shell commands in a file and executing them as a script.

For example, instead of manually checking disk space every day, you can create a script that checks disk usage and reports when it crosses a threshold. This is especially useful for DevOps engineers because many administration and deployment tasks are repetitive.

INTERVIEW TRAP QUESTION

Why does a script work with `bash script.sh` but fail with `sh script.sh` on Ubuntu?

On Ubuntu, `/bin/sh` points to **dash**, not Bash. Bash-only features such as `[[ ]]`, arrays and `{1..5}` fail in dash. Use the correct shebang and run Bash scripts with Bash.

2 Creating and Executing Your First Bash Script

```
# Create a script
nano hello.sh
```

Add:

```
#!/bin/bash

echo "Hello, Linux!"
echo "Welcome to Bash scripting"
```

Understanding the Shebang

The first line, `#!/bin/bash`, is the shebang. It tells the operating system which interpreter should execute the script. `#!/usr/bin/env bash` is a more portable alternative that finds Bash using PATH.

```
# Give execution permission
chmod +x hello.sh

# Execute the script
./hello.sh

# Alternatively
bash hello.sh
```

Three Ways to Run a Script

Method	Behavior
`./script.sh`	Needs execute permission; runs in a new process using the shebang
`bash script.sh`	No execute permission needed, because Bash reads the file directly
`source script.sh` (or `. script.sh`)	Runs in the **current** shell, so variables and directory changes remain after it finishes

`source` is typically used to load environment variables, for example `source ~/.bashrc`.

3 Understanding Variables

Variables store values that can be reused throughout a script.

```
#!/bin/bash

name="DevOps Engineer"
company="ABC Technologies"

echo "Role: $name"
echo "Company: $company"
```

Output:

```
Role: DevOps Engineer
Company: ABC Technologies
```

Important Variable Rules

- Do not put spaces around `=` when assigning a variable.
- Use `$variable` to access its value.
- Variable names are case-sensitive.

```
# Correct
name="Linux"

# Incorrect (Bash treats "name" as a command)
name = "Linux"
```

Command Substitution

You can store the output of a command in a variable.

```
current_date=$(date)
hostname=$(hostname)

echo "Date: $current_date"
echo "Server: $hostname"
```

Single vs Double Quotes

```
name="Ravi"

echo "Hello $name"     # Hello Ravi   (double quotes expand variables)
echo 'Hello $name'     # Hello $name  (single quotes are literal)
```

Always quote variables, for example `"$file"`. Without quotes, values containing spaces are split into multiple words.

Environment Variables

Environment variables are values made available to processes.

```
# View environment variables
echo $HOME
echo $USER
echo $PATH

# Create a shell variable
APP_ENV="production"

# Export it so child processes can access it
export APP_ENV="production"

# List environment variables
printenv
```

Special Variables

Variable	Meaning
`$0`	Script name
`$1`, `$2` ...	Positional arguments
`$#`	Number of arguments
`$@`	All arguments, each kept separate when quoted
`$*`	All arguments, joined into one string when quoted
`$?`	Exit status of the last command
`$$`	PID of the current script
`$!`	PID of the last background process

Useful Parameter Expansion

Syntax	Meaning
`${VAR:-default}`	Use `default` if VAR is unset or empty
`${VAR:?message}`	Stop the script with `message` if VAR is unset or empty
`${#VAR}`	Length of the value
`${FILE##*/}`	Filename without its path (like `basename`)
`${FILE%.*}`	Value with the file extension removed

```
# Use "staging" if no first argument is given
ENVIRONMENT=${1:-staging}
```

4 Taking User Input

Bash provides the `read` command to accept input from users.

```
#!/bin/bash

echo "Enter your name:"
read name

echo "Welcome, $name"
```

A shorter version:

```
read -p "Enter your name: " name
echo "Welcome, $name"
```

Taking Input Without Displaying It

This is useful for password input:

```
read -s -p "Enter password: " password
echo
echo "Password received"
```

Avoid printing passwords or other secrets in logs.

5 Understanding Conditional Statements

Conditional statements allow scripts to make decisions. Bash commonly uses `if`, `elif` and `else`.

Basic if Statement

```
#!/bin/bash

disk_usage=75

if [ "$disk_usage" -gt 80 ]; then
    echo "Warning: Disk usage is high"
else
    echo "Disk usage is normal"
fi
```

if-elif-else Statement

```
#!/bin/bash

cpu_usage=90

if [ "$cpu_usage" -gt 90 ]; then
    echo "Critical CPU usage"
elif [ "$cpu_usage" -gt 70 ]; then
    echo "High CPU usage"
else
    echo "CPU usage is normal"
fi
```

Numeric Comparison Operators

Operator	Meaning
`-eq`	Equal to
`-ne`	Not equal to
`-gt`	Greater than
`-lt`	Less than
`-ge`	Greater than or equal
`-le`	Less than or equal

String Operators

Operator	Meaning
`=`	Strings are equal
`≠`	Strings are different
`-z`	String is empty
`-n`	String is not empty

File Condition Checks

Condition	Meaning
`-f`	Regular file exists
`-d`	Directory exists
`-e`	File or directory exists
`-r`	Read permission exists
`-w`	Write permission exists
`-x`	Execute permission exists

```
#!/bin/bash

if [ -f "/etc/passwd" ]; then
    echo "File exists"
else
    echo "File not found"
fi
```

[ ] vs [[ ]]

Feature	`[ ]`	`[[ ]]`
Type	Command (POSIX, works in any shell)	Bash keyword
Unquoted empty variables	Can cause errors	Handled safely
Pattern matching	No	Yes, for example `[[ $file == *.log ]]`
`&&` and `||` inside	No	Yes

In Bash scripts, `[[ ]]` is generally safer. Use `[ ]` when the script must run in `sh`.

case Statement

`case` is cleaner than many `if-elif` blocks, and is often used for start/stop scripts.

```
case "$1" in
    start)   systemctl start nginx ;;
    stop)    systemctl stop nginx ;;
    restart) systemctl restart nginx ;;
    *)       echo "Usage: $0 {start|stop|restart}"; exit 1 ;;
esac
```

6 Understanding Loops

Loops allow commands to execute repeatedly.

for Loop

A `for` loop is useful when iterating over a list.

```
#!/bin/bash

for server in server1 server2 server3
do
    echo "Checking $server"
done

# Loop over a range of numbers
for i in {1..5}
do
    echo "Attempt $i"
done
```

while Loop

A `while` loop runs while a condition remains true.

```
#!/bin/bash

count=1

while [ "$count" -le 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```

`$(( ))` performs arithmetic. Bash supports integer arithmetic only.

Reading a File Line by Line

```
while IFS= read -r line
do
    echo "$line"
done < servers.txt
```

INTERVIEW INSIGHT

Use `while read` to read files. `for line in $(cat file)` splits lines on spaces, so a line such as `web server 1` becomes three separate items.

Loop Through Files

```
#!/bin/bash

for file in /var/log/*.log
do
    [ -f "$file" ] || continue
    echo "Log file: $file"
done
```

Arrays

```
SERVERS=("web1" "web2" "web3")

echo "${SERVERS[0]}"        # First element
echo "${#SERVERS[@]}"       # Number of elements

for server in "${SERVERS[@]}"
do
    echo "$server"
done
```

break and continue

`break` stops a loop:

```
for number in 1 2 3 4 5
do
    if [ "$number" -eq 3 ]; then
        break
    fi
    echo "$number"
done
```

`continue` skips the current iteration:

```
for number in 1 2 3 4 5
do
    if [ "$number" -eq 3 ]; then
        continue
    fi
    echo "$number"
done
```

7 Understanding Functions

Functions allow you to reuse a block of commands instead of writing the same code repeatedly.

```
#!/bin/bash

check_service() {
    if systemctl is-active --quiet nginx; then
        echo "Nginx is running"
    else
        echo "Nginx is not running"
    fi
}

check_service
```

Functions with Arguments

```
#!/bin/bash

greet() {
    echo "Hello, $1"
}

greet "DevOps Engineer"
```

Here, `$1` represents the first argument passed to the function.

Variable	Meaning
`$1`	First argument
`$2`	Second argument
`$@`	All arguments
`$#`	Number of arguments

```
#!/bin/bash

echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "Total arguments: $#"
```

```
./script.sh production nginx
```

Returning Data from a Function

```
get_disk_usage() {
    local mount="$1"
    df -P "$mount" | awk 'NR==2 {gsub("%","",$5); print $5}'
}

usage=$(get_disk_usage /)
echo "Root usage: ${usage}%"
```

- `local` keeps a variable inside the function.
- `return` sets an **exit code** (0–255) only; it cannot return text.
- To return data, `echo` it and capture it using `$( )`.

8 Understanding Exit Codes

Exit codes indicate whether a command or script completed successfully.

- `0` generally means success.
- A non-zero value generally indicates failure.

```
# Successful command
ls /etc
echo $?

# Failing command
ls /invalid-directory
echo $?
```

Using Exit Codes in Scripts

```
#!/bin/bash

mkdir /tmp/test-folder

if [ $? -eq 0 ]; then
    echo "Directory created successfully"
else
    echo "Directory creation failed"
fi
```

A cleaner approach is to use the command directly in an `if` condition:

```
if mkdir -p /tmp/test-folder; then
    echo "Directory created successfully"
else
    echo "Directory creation failed"
fi
```

&& and ||

```
mkdir -p /backup && echo "Created" || echo "Failed"
```

- `&&` runs the next command only if the previous one **succeeded**.
- `||` runs the next command only if the previous one **failed**.

> **INTERVIEW CAUTION**
>
> `A && B || C` is not a true if/else. If `B` fails, `C` also runs. Use a real `if` statement when it matters.

Exit from a Script

```
#!/bin/bash

if [ ! -f "/etc/passwd" ]; then
    echo "Required file missing"
    exit 1
fi

echo "Validation successful"
exit 0
```

> **DEVOPS NOTE**
>
> Exit codes are extremely important in CI/CD pipelines. Jenkins, GitLab CI and other automation tools use command exit status to determine whether a build or deployment step succeeded or failed.

9 Error Handling and Debugging

Production scripts should handle errors properly rather than silently continuing after failures.

Strict Mode

```
#!/bin/bash
set -Eeuo pipefail
```

Option	Meaning
`-e`	Exit when a command fails
`-u`	Treat unset variables as an error
`-o pipefail`	A pipeline fails if *any* command in it fails, not only the last one
`-E`	Make `ERR` traps also work inside functions

```
# set -u example: exits with an error if APP_NAME is not defined
#!/bin/bash
set -u
echo "$APP_NAME"

# pipefail example: the failed cat is reported, not hidden by grep
#!/bin/bash
set -o pipefail
cat /invalid/file | grep "ERROR"
```

INTERVIEW CAUTION

`set -e` has edge cases. It does not stop the script for commands used in `if` conditions, or commands followed by `&&` or `||`. Strict mode is not a substitute for proper error handling; some commands intentionally return non-zero statuses, so handle expected failures explicitly.

Protecting Destructive Commands

```
# DANGEROUS: if BACKUP_DIR is empty, this becomes rm -rf /*
rm -rf "$BACKUP_DIR"/*

# Safe: stops the script if BACKUP_DIR is unset or empty
rm -rf "${BACKUP_DIR:?BACKUP_DIR is not set}"/*
```

trap – Cleanup on Exit

`trap` runs a command when the script exits, fails or receives a signal.

```
TMP_FILE=$(mktemp)

# Always remove the temporary file, even if the script fails
trap 'rm -f "$TMP_FILE"' EXIT

# Report the line number where an error occurred
trap 'echo "Error on line $LINENO"' ERR
```

Common uses: deleting temporary files, removing lock files, and handling Ctrl+C (`INT`) and `TERM` gracefully.

Debugging a Script

Command	Purpose
`bash -n script.sh`	Check syntax without running the script
`bash -x script.sh`	Print each command as Bash executes it
`set -x` / `set +x`	Turn tracing on and off inside a script
`shellcheck script.sh`	Find bugs and bad practices before running

> **WARNING**
>
> Avoid enabling debug output when scripts may expose passwords, tokens or other secrets.

10 Working with Command-Line Arguments

Arguments allow users and automation tools to pass values to scripts.

```
#!/bin/bash

environment=$1
application=$2

echo "Environment: $environment"
echo "Application: $application"
```

```
./deploy.sh production myapp
```

```
Environment: production
Application: myapp
```

Validate Required Arguments

```
#!/bin/bash

if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <environment> <application>"
    exit 1
fi

environment=$1
application=$2

echo "Deploying $application to $environment"
```

This prevents the script from running without the required inputs.

11 Redirection and Logging

Syntax	Meaning
`>`	Write output to a file (overwrite)
`>>`	Append output to a file
`2>`	Redirect errors
`2>&1`	Send errors to the same place as normal output
`&> /dev/null`	Discard all output
`| tee -a file`	Show output on screen and append it to a file

INTERVIEW TRAP QUESTION

Are `cmd > file 2>&1` and `cmd 2>&1 > file` the same?

No. Redirections are processed left to right. Only the first form sends **both** output and errors to the file. The second sends errors to the terminal, because `2>&1` is applied before output is redirected.

A Simple Logging Function

```
LOG_FILE="/var/log/myscript.log"

log() {
    echo "$(date '+%F %T') [$1] ${*:2}" | tee -a "$LOG_FILE"
}

log INFO "Backup started"
log ERROR "Backup failed"
```

Here-Documents: Writing Multi-Line Files

```
cat <<EOF > /opt/myapp/config.env
APP_ENV=production
APP_PORT=3000
EOF

# Writing to a root-owned file: use sudo tee, not sudo cat >
sudo tee /etc/myapp/config.env > /dev/null <<EOF
APP_ENV=production
EOF
```

Use `<<'EOF'` (quoted) if the content contains `$` characters that must not be expanded.

12 Practical DevOps Automation Scripts

These examples demonstrate how Bash can automate common server administration tasks.

Script 1: Disk Usage Monitoring

Check whether disk usage crosses a defined threshold.

```
#!/bin/bash

THRESHOLD=80

usage=$(df -P / | awk 'NR==2 {gsub("%","",$5); print $5}')

echo "Current disk usage: ${usage}%"

if [ "$usage" -ge "$THRESHOLD" ]; then
    echo "WARNING: Disk usage is above ${THRESHOLD}%"
else
    echo "Disk usage is normal"
fi
```

This script checks the filesystem containing `/`. `df -P` keeps each filesystem on one line, which makes the output safe to parse.

Script 2: Service Health Check

```
#!/bin/bash

SERVICE="nginx"

if systemctl is-active --quiet "$SERVICE"; then
    echo "$SERVICE is running"
else
    echo "$SERVICE is not running"
    exit 1
fi
```

This can be used in scheduled checks or monitoring workflows.

Script 3: Application Backup with Retention

```
#!/bin/bash
set -euo pipefail

SOURCE_PARENT="/opt"
APP_NAME="myapp"
BACKUP_DIR="/backup"
DATE=$(date +%F_%H-%M-%S)

mkdir -p "$BACKUP_DIR"

if tar -czf "$BACKUP_DIR/${APP_NAME}_$DATE.tar.gz" -C "$SOURCE_PARENT" "$APP_NAME"; then
    echo "Backup completed successfully"
else
    echo "Backup failed"
    exit 1
fi

# Keep only the last 7 days of backups
find "$BACKUP_DIR" -name "${APP_NAME}_*.tar.gz" -mtime +7 -delete
```

In production, also consider available disk space, file consistency, permissions and restoring backups regularly to verify they work.

Script 4: Check Multiple Servers

```
#!/bin/bash

SERVERS=("192.168.1.10" "192.168.1.20" "192.168.1.30")

for server in "${SERVERS[@]}"
do
    if ping -c 1 -W 2 "$server" > /dev/null 2>&1; then
        echo "$server is reachable"
    else
        echo "$server is not responding to ping"
    fi
done
```

A failed ping does not always mean the server is unavailable because ICMP may be blocked. For application health checks, test the actual service endpoint or port.

Script 5: Application Deployment Preparation

```
#!/bin/bash
set -Eeuo pipefail

APP_DIR="/opt/myapp"
APP_USER="appuser"

if ! id "$APP_USER" > /dev/null 2>&1; then
    echo "Application user does not exist"
    exit 1
fi

mkdir -p "$APP_DIR"
chown "$APP_USER:$APP_USER" "$APP_DIR"

echo "Application directory is ready"
```

This is a preparation example, not a complete deployment pipeline. A real deployment should also handle application artifacts, configuration, service health checks and rollback.

Script 6: Restart a Service If It Is Down

```
#!/bin/bash

SERVICE="nginx"
LOG_FILE="/var/log/service-check.log"

if ! systemctl is-active --quiet "$SERVICE"; then
    echo "$(date '+%F %T') $SERVICE is down, restarting" >> "$LOG_FILE"
    systemctl restart "$SERVICE"
fi
```

In production, systemd's `Restart=on-failure` is usually preferred. This script shows that you understand the logic.

Script 7: Delete Logs Older Than 7 Days

```
# First, preview which files would be deleted
find /var/log/myapp -type f -name "*.log" -mtime +7 -print

# Then delete them
find /var/log/myapp -type f -name "*.log" -mtime +7 -delete
```

INTERVIEW LINE

"I explain the logic first, then write the script, then mention edge cases." Log-parsing scripts such as finding the top IP addresses are covered in Chapter 11.

13 Scheduling Shell Scripts Using Cron

Once a script is created, you can schedule it to run automatically using cron.

```
# Edit the current user's cron jobs
crontab -e
```

```
# Run a disk monitoring script every 10 minutes
*/10 * * * * /opt/scripts/disk_check.sh

# Run a backup script every day at 2 AM
0 2 * * * /opt/scripts/backup.sh

# Redirect output to a log
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

Cron is useful for recurring tasks such as:

- Backups.
- Log cleanup.
- Health checks.
- Disk monitoring.
- Report generation.

INTERVIEW TRAP QUESTION

A script works when run manually but fails in cron. Why?

Cron runs with a minimal environment: a short PATH, no `.bashrc`, and a different working directory. Use absolute paths, set required variables inside the script, escape `%` characters in the crontab (see Chapter 5), and redirect output to a log so you can see the error.

Preventing Overlapping Runs

If a job takes longer than its interval, two copies can run at the same time. `flock` prevents this:

```
*/10 * * * * flock -n /tmp/disk_check.lock /opt/scripts/disk_check.sh
```

With `-n`, a new run exits immediately if the previous run still holds the lock.

For more complex workflows, dependency handling and service-style scheduled jobs, systemd timers are another useful option (see Chapter 5).

14 Bash Scripting Best Practices

When writing scripts for production environments, follow these practices:

1. Use a proper shebang such as `#!/bin/bash`.
2. Use meaningful variable names.
3. Quote variables, especially when they contain paths or user input.
4. Validate required arguments.
5. Check command exit codes.
6. Handle expected errors explicitly.
7. Use absolute paths in scheduled scripts.
8. Write useful logs with timestamps.
9. Avoid hardcoding passwords and API tokens.
10. Use least-privilege permissions.
11. Test scripts in non-production environments, and run `shellcheck`.
12. Make scripts safe to run more than once where possible.

Why Quote and Validate Variables?

```
rm -rf $APP_DIR/*
```

If the variable is empty or contains unexpected content, this command can be dangerous. Quoting alone does not fix it: if `APP_DIR` is empty, `rm -rf -- "$APP_DIR"/*` still becomes `rm -rf -- /*`. Validate the variable as well:

```
rm -rf -- "${APP_DIR:?APP_DIR is not set}"/*
```

Even then, confirm the target directory and understand exactly what will be deleted before running such commands.

What Is Idempotency?

An idempotent script gives the same result no matter how many times it runs. This matters because automation often reruns scripts.

```
# Not idempotent: adds the line again on every run
echo "vm.swappiness=10" >> /etc/sysctl.conf

# Idempotent: adds the line only if it is missing
grep -qxF "vm.swappiness=10" /etc/sysctl.conf || echo "vm.swappiness=10" >> /etc/sysctl.c
```

Commands such as `mkdir -p` are already idempotent.

15 Shell Scripting Troubleshooting Scenarios

Scenario 1 Script works manually but fails in cron

- Cron has a different, minimal PATH and environment.
- Use absolute paths for commands and files.
- Redirect output to a log file to see the actual error.

Scenario 2 Script deleted the wrong files

- A variable used in a path was empty or unset.
- Use `set -u` and `${VAR:?}`.
- Always do a dry run (for example, `find ... -print`) before deleting.

Scenario 3 Script reports success, but a step actually failed

- A failure inside a pipeline was hidden because only the last command's exit code was checked.
- Add `set -o pipefail` and check exit codes.

Scenario 4 Error: "/bin/bash^M: bad interpreter"

The script was created or edited on Windows and has CRLF line endings.

```
dos2unix script.sh

# Or, without dos2unix
sed -i 's/\r$//' script.sh
```

Scenario 5 **"Permission denied" when running a script**

- Execute permission is missing: run `chmod +x script.sh`.
- The filesystem is mounted with `noexec` (common on `/tmp`). Run it with `bash script.sh`, or move the script to another location.

16 Common Interview Questions

Q1. What is Bash scripting?

Bash scripting is writing a sequence of shell commands in a file so they can be executed together to automate tasks.

Q2. What is a shebang?

A shebang identifies the interpreter used to execute a script, for example `#!/bin/bash`.

Q3. What is the difference between a shell variable and an environment variable?

A shell variable belongs to the current shell. An exported environment variable is also available to child processes.

Q4. What does `$?` represent?

It represents the exit status of the most recently executed command.

Q5. What is the difference between `$1` and `$@`?

`$1` represents the first positional argument. `$@` represents all positional arguments.

Q6. What is the difference between `for` and `while` loops?

A `for` loop commonly iterates over a list of values. A `while` loop continues while a specified condition is true.

Q7. What is the purpose of `chmod +x`?

It adds executable permission to a file.

Q8. What does `set -e` do?

It causes a Bash script to exit when certain commands fail, instead of continuing blindly.

Q9. What is the purpose of `set -u`?

It treats references to unset variables as errors.

Q10. What is `pipefail`?

It makes a pipeline report failure when a command within the pipeline fails, rather than only considering the last command.

Q11. How do you debug a Bash script?

```
bash -x script.sh
```

Also use `bash -n` to check syntax and `shellcheck` to find common bugs.

Q12. How do you schedule a script in Linux?

Use cron with `crontab -e` to define its execution schedule, or use a systemd timer.

Q13. Why are exit codes important in CI/CD?

CI/CD tools use exit codes to determine whether commands and pipeline steps succeeded or failed.

Q14. How would you write a script to check disk usage?

Use `df` to collect usage, extract the percentage using a tool such as `awk`, compare it with a threshold and return an appropriate status.

Q15. Why should scripts use absolute paths?

Scheduled jobs and automation tools may have a different working directory and environment from an interactive terminal. Absolute paths make execution more predictable.

Q16. What is the difference between `"$@"` and `"$*"`?

`"$@"` keeps each argument as a separate word. `"$*"` joins all arguments into a single string.

Q17. What is the difference between `[ ]` and `[[ ]]`?

`[ ]` is a POSIX command that works in any shell. `[[ ]]` is a Bash keyword that handles empty variables safely and supports pattern matching and `&&` / `||` inside the test.

Q18. What is the difference between single and double quotes?

Single quotes treat everything literally. Double quotes expand variables and command substitutions.

Q19. What is the difference between `source script.sh` and `./script.sh`?

`source` runs the script in the current shell, so its variables remain afterwards. `./script.sh` runs it in a separate child process.

Q20. Why might a script work manually but fail in cron?

Cron uses a minimal environment and PATH and a different working directory. Use absolute paths, define variables inside the script and redirect output to a log.

Q21. What does `trap` do?

It runs a command when the script exits, fails or receives a signal. It is commonly used to clean up temporary files and lock files.

Q22. What does the error "/bin/bash^M: bad interpreter" mean?

The script has Windows (CRLF) line endings. Convert it using `dos2unix` or `sed -i 's/\r$//'`.

Q23. What is the difference between `cmd > file 2>&1` and `cmd 2>&1 > file`?

Redirections are applied left to right. The first sends both output and errors to the file. The second sends only normal output to the file and errors to the terminal.

Q24. How do you stop a cron job from overlapping with its previous run?

Run it through `flock -n /path/to/lockfile`, so a new run exits immediately if the previous run is still active.

17 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ Linux shells, Bash, and the sh vs bash difference.
- ✓ Creating and executing shell scripts, and `source` vs `./`.
- ✓ Shebang and executable permissions.

- ✓ Variables, quoting, special variables and parameter expansion.
- ✓ User input and command substitution.
- ✓ Conditional statements, `[[ ]]` and `case`.
- ✓ for and while loops, reading files and arrays.
- ✓ Functions, `local` and returning data.
- ✓ Exit codes and error handling, including strict mode and `trap`.
- ✓ Debugging using `bash -x`, `bash -n` and `shellcheck`.
- ✓ Redirection, logging and here-documents.
- ✓ Automation scripts for monitoring, backups and services.
- ✓ Scheduling scripts using cron and preventing overlapping runs.
- ✓ Bash scripting best practices and idempotency for production.

PRACTICAL EXERCISES

1. Create a Bash script that displays the current date, username and hostname.
2. Create a script that accepts a username as input.
3. Write an `if-else` condition to check whether a file exists.
4. Write a loop that displays numbers from 1 to 10.
5. Create a function that checks whether Nginx is running.
6. Write a script that accepts environment and application names as arguments.
7. Create a disk usage monitoring script with an 80% threshold.
8. Create a backup script that generates timestamped archives and keeps only 7 days.
9. Write a script that checks the availability of three servers.
10. Schedule a script using cron and verify its execution through logs.
11. Test how exit codes behave when a command succeeds or fails.
12. Debug a script using `bash -x`.
13. Run `shellcheck` on one of your scripts and fix its warnings.
14. Write a script that uses `trap` to remove a temporary file, even when interrupted with Ctrl+C.
15. Prevent a cron job from overlapping using `flock`.

NEXT CHAPTER

Chapter 11: Text Processing and Command-Line Power Tools

You will learn `grep`, `awk`, `sed`, `cut`, `sort`, `uniq`, `xargs`, `find` and other commands used to analyze logs, process large files and troubleshoot Linux systems efficiently.

END OF CHAPTER 10